Agenda

- Wątki
- Współdzielone zasoby
- Pule wątków

Wątki

Program sekwencyjny

```
public class Counter {

    public void run() {
        for(int i=0; i<10; ++i)
            System.out.print(i);
    }
}


Counter counter = new Counter();
counter.run();
```

```
0123456789
```

Wątek

```
public class Counter extends Thread {
    @Override
    public void run() {
        for(int i=0; i<10; ++i)
            System.out.print(i);
    }
}


Counter counter = new Counter();
counter.start();
```

```
0123456789
```

Równoległe uruchomienie wątków

```
public class Counter extends Thread {
    @Override
    public void run() {
        for(int i=0; i<10; ++i)
            System.out.print(i);
    }
}
```

```
Counter counter1 = new Counter();
Counter counter2 = new Counter();

counter1.start();
counter2.start();
```

```
0123456701234567898
0123456012345678899
0123456012345677898
0123456701234567898
0123456780123456789
```

Interfejs Runnable

```
public class Counter implements Runnable {
    @Override
    public void run() {
        for(int i=0; i<10; ++i)
            System.out.print(i);
    }
}

Counter counter1 = new Counter();
Counter counter2 = new Counter();
Thread counterThread1 = new Thread(counter1);
Thread counterThread2 = new Thread(counter2);

counterThread1.start();
counterThread2.start();
```

```
0123456701234567898
0123456012345678999
0123456012345678998
0123456701234567898
0123456780123456789
```

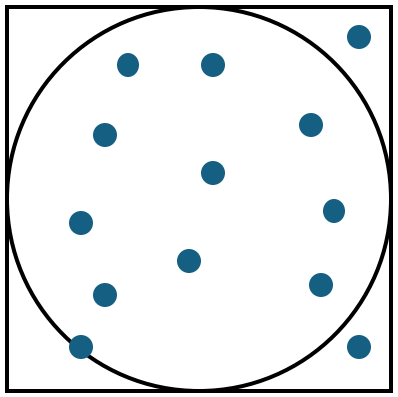
Runnable pozwala na dziedziczenie

```
public class Data {
    protected int[] data;

    public Data(int size, int lowBound, int highBound) {
        data = new Random().ints(size, lowBound, highBound).toArray();
    }
}

public class DataReader extends Data implements Runnable {
    public DataReader(int size, int lowBound, int highBound) {
        super(size, lowBound, highBound);
    }

    @Override
    public void run() {
        for(int i=0; i<data.length; ++i)
            System.out.print(data[i]);
    }
}
```

```
DataReader dataReader = new DataReader(10, 0, 10);
Thread dataReaderThread = new Thread(dataReader);
dataReaderThread.start();
```

Obliczenie liczby π metodą Monte Carlo

```
public class CalculatePi {
    private double pi;
    public double getPi() { return pi;}
    public double error() { return Math.abs(pi-Math.PI); }

    public void run() {
        long inside = 0; long total = 0; pi = 0;

        for(int i=0; i<1000; ++i) {
            double x = Math.random();
            double y = Math.random();

            if (x * x + y * y <= 1) inside++;
            total++;
        }
        pi = 4.0 * inside / total;
    }
}
```



```
CalculatePi cpi = new CalculatePi();
cpi.run();
System.out.println(cpi.getPi());
System.out.println(cpi.error());
```

```
3.208
0.06640734641020707
```

Przeniesienie obliczeń do wątku

```java
public class CalculatePi implements Runnable {
    /* ... */
    private boolean stop = false;
    public void stopCalculation() { stop = true; }

    @Override
    public void run() {
        long inside = 0; long total = 0; pi = 0;

        while (!stop) {
            double x = Math.random();
            double y = Math.random();

            if (x * x + y * y <= 1) inside++;
            total++;
        }
        pi = 4.0 * inside / total;
    }
}
```

```java
CalculatePi cpi = new CalculatePi();
Thread cpiThread = new Thread(cpi);
cpiThread.start();
System.out.println(cpi.getPi());
System.out.println(cpi.error());
```

```
0.0
3.141592653589793

Program kontynuuje działanie
```

Interaktywność programu

```java
public class CalculatePi implements Runnable {
    /* ... */
    private boolean stop = false;
    public void stopCalculation() { stop = true; }

    @Override
    public void run() {
        long inside = 0; long total = 0; pi = 0;

        while (!stop) {
            double x = Math.random();
            double y = Math.random();

            if (x * x + y * y <= 1) inside++;
            total++;
        }
        pi = 4.0 * inside / total;
    }
}
```

```java
CalculatePi cpi = new CalculatePi();
Thread cpiThread = new Thread(cpi);

Scanner scanner = new Scanner(System.in);
cpiThread.start();
scanner.nextLine();
cpi.stopCalculation();
System.out.println(cpi.getPi());
System.out.println(cpi.error());
```

```
0.0
2.409077971288198 2E-4
```

Zaczekanie na zakończenie wątku

```
public class CalculatePi implements Runnable {
    /* ... */
    private boolean stop = false;
    public void stopCalculation() { stop = true; }

    @Override
    public void run() {
        long inside = 0; long total = 0; pi = 0;

        while (!stop) {
            double x = Math.random();
            double y = Math.random();

            if (x * x + y * y <= 1) inside++;
            total++;
        }
        pi = 4.0 * inside / total;
    }
}
```

```
CalculatePi cpi = new CalculatePi();
Thread cpiThread = new Thread(cpi);

Scanner scanner = new Scanner(System.in);
cpiThread.start();
scanner.nextLine();
cpi.stopCalculation();
cpiThread.join();

System.out.println(cpi.getPi());
System.out.println(cpi.error());
```

```
3.1416886889726734
9.603538288027735E-5
```

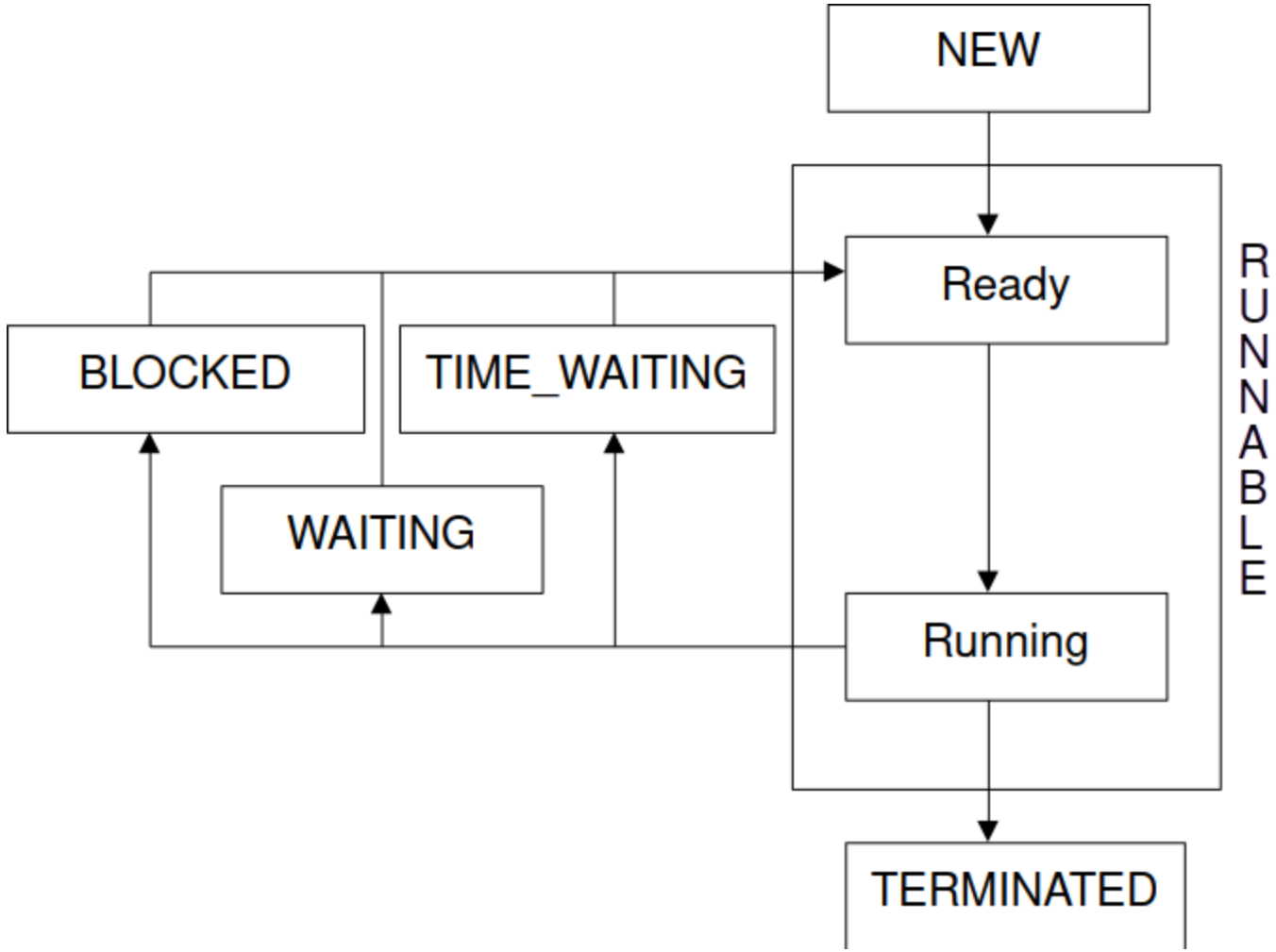
Ponowne uruchomienie wątku

```
CalculatePi cpi = new CalculatePi();
Thread cpiThread = new Thread(cpi);
Scanner scanner = new Scanner(System.in);

for (int i = 0; i < 2; ++i) {
    cpiThread.start();
    scanner.nextLine();
    cpi.stopCalculation();
    cpiThread.join();
    System.out.println(cpi.getPi());
    System.out.println(cpi.error());
}
```

```
3.141441560865637
1.510927241561432E-4
Exception in thread "main" java.lang.IllegalThreadStateException
    at java.base/java.lang.Thread.start(Thread.java:1512)
    at Main.main(Main.java:6)
```

Stany wątku

- NEW
- RUNNABLE
- BLOCKED
- WAITING
- TIMED_WAITING
- TERMINATED



Odczyt stanu wątku

```
CalculatePi cpi = new CalculatePi();
Thread cpiThread = new Thread(cpi);
System.out.println(cpiThread.getState());
Scanner scanner = new Scanner(System.in);
cpiThread.start();
System.out.println(cpiThread.getState());
scanner.nextLine();
cpi.stopCalculation();
cpiThread.join();
System.out.println(cpiThread.getState());
 System.out.println(cpi.getPi());
System.out.println(cpi.error());
```

```
NEW
RUNNABLE
TERMINATED
3.141589696054165
2.957535627956531E-6
```

Współdzielone zasoby

Inkrementacja fragmentu tablicy

```
class Worker implements Runnable {
    private final int[] array;
    private final int startIndex;
    private final int endIndex;

    public Worker(int[] array, int startIndex, int endIndex) {
        this.array = array;
        this.startIndex = startIndex;
        this.endIndex = endIndex;
    }

    @Override
    public void run() {
        for (int i = startIndex; i < endIndex; i++)
            array[i]++;
   }
}
```

Podział zadania na wątki

```
int numThreads = 3;
int[] ints = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
Thread[] threads = new Thread[numThreads];
int chunkSize = ints.length / numThreads;
for (int i = 0; i < numThreads; i++) {
    int startIndex = i * chunkSize;
    int endIndex = (i == numThreads - 1) ? ints.length : (i + 1) * chunkSize;

    threads[i] = new Thread(new Worker(ints, startIndex, endIndex));
    threads[i].start();
}
for (Thread thread : threads) thread.join();
Arrays.stream(ints).forEach(System.out::print);
```

```
12345678910
```

Liczba rdzeni

```
int numThreads = Runtime.getRuntime().availableProcessors();
int[] ints = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
Thread[] threads = new Thread[numThreads];
int chunkSize = ints.length / numThreads;
for (int i = 0; i < numThreads; i++) {
    int startIndex = i * chunkSize;
    int endIndex = (i == numThreads - 1) ? ints.length : (i + 1) * chunkSize;

    threads[i] = new Thread(new Worker(ints, startIndex, endIndex));
    threads[i].start();
}
for (Thread thread : threads) thread.join();
Arrays.stream(ints).forEach(System.out::print);
```

```
12345678910
```

Czasochłonne obliczenia – liczby pierwsze

```
@Override
public void run() {
   for (int i = startIndex; i < endIndex; i++) array[i] = sumOfPrimes(i * 1000, (i + 1) * 1000);
}

private int sumOfPrimes(int start, int end) {
   int sum = 0;
   for (int num = start; num < end; num++)
       if (isPrime(num))
           sum += num;
   return sum;
 }

private boolean isPrime(int num) {
   if (num <= 1) return false;
   for (int i = 2; i <= Math.sqrt(num); i++)
       if (num % i == 0) return false;
   return true;
}
```

Czas w zależności od liczby wątków (8 rdzeni)

Wątki	Czas [ms]	% 1 wątku	Wątki	Czas [ms]	% 1 wątku
1	61927	1,00	13	9736	0,16
2	39053	0,63	14	9350	0,15
3	27486	0,44	15	9356	0,15
4	21314	0,34	16	9141	0,15
5	17351	0,28	17	9003	0,15
6	14561	0,24	18	9087	0,15
7	12632	0,20	19	8785	0,14
8	11159	0,18	20	9164	0,15
9	11381	0,18	21	8475	0,14
10	10973	0,18	22	8731	0,14
11	10297	0,17	23	8814	0,14
12	10250	0,17	24	8631	0,14

Współdzielona zmienna

```
class Worker implements Runnable {
    static private int sum = 0;
    private final int[] array;
    private final int startIndex;
    private final int endIndex;

    public static int getSum() {
        return sum;
    }

    public Worker(int[] array, int startIndex, int endIndex) {
        this.array = array; this.startIndex = startIndex; this.endIndex = endIndex;
    }

    @Override
    public void run() {
        for (int i = startIndex; i < endIndex; i++)
            sum += array[i];
    }
}
```

Suma zawartości tablicy

```
int numThreads = Runtime.getRuntime().availableProcessors();
int[] ints = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
Thread[] threads = new Thread[numThreads];
int chunkSize = ints.length / numThreads;
for (int i = 0; i < numThreads; i++) {
    int startIndex = i * chunkSize;
    int endIndex = (i == numThreads - 1) ? ints.length : (i + 1) * chunkSize;

    threads[i] = new Thread(new Worker(ints, startIndex, endIndex));
    threads[i].start();
}
for (Thread thread : threads) thread.join();
System.out.println(Worker.getSum());
```

Większa tablica

```
int numThreads = Runtime.getRuntime().availableProcessors();
int[] ints = new int[10000];
Arrays.fill(ints, 1);
Thread[] threads = new Thread[numThreads];
int chunkSize = ints.length / numThreads;
for (int i = 0; i < numThreads; i++) {
    int startIndex = i * chunkSize;
    int endIndex = (i == numThreads - 1) ? ints.length : (i + 1) * chunkSize;

    threads[i] = new Thread(new Worker(ints, startIndex, endIndex));
    threads[i].start();
}
for (Thread thread : threads) thread.join();
System.out.println(Worker.getSum());
```

```
10000
```

Jeszcze większa tablica

```
int numThreads = Runtime.getRuntime().availableProcessors();
int[] ints = new int[100000];
Arrays.fill(ints, 1);
Thread[] threads = new Thread[numThreads];
int chunkSize = ints.length / numThreads;
for (int i = 0; i < numThreads; i++) {
    int startIndex = i * chunkSize;
    int endIndex = (i == numThreads - 1) ? ints.length : (i + 1) * chunkSize;

    threads[i] = new Thread(new Worker(ints, startIndex, endIndex));
    threads[i].start();
}
for (Thread thread : threads) thread.join();
System.out.println(Worker.getSum());
```

```
47382
```

Synchronizacja metody statycznej

```
class Worker implements Runnable {
    static private int sum = 0;
    private final int[] array;
    private final int startIndex, endIndex;
    public static int getSum() { return sum.get(); }
    public Worker(int[] array, int startIndex, int endIndex) {
        this.array = array; this.startIndex = startIndex; this.endIndex = endIndex;
    }

    static synchronized private void add(int variable) { sum += variable; }

    @Override
    public void run() {
        for (int i = startIndex; i < endIndex; i++)
        add(array[i]);
    }
}
```

```
100000
```

Synchronizacja bloku

```
class Worker implements Runnable {
    static private int sum = 0;
    private final int[] array;
    private final int startIndex, endIndex;
    public static int getSum() { return sum.get(); }
    public Worker(int[] array, int startIndex, int endIndex) {
        this.array = array; this.startIndex = startIndex; this.endIndex = endIndex;
    }

    @Override
    public void run() {
        for (int i = startIndex; i < endIndex; i++)
            synchronized (Worker.class) {
                sum += array[i];
            }
}

}
```

```
100000
```

Typ atomowy

```
class Worker implements Runnable {
    static private AtomicInteger sum = new AtomicInteger(0);
    private final int[] array;
    private final int startIndex, endIndex;

    public static int getSum() { return sum.get(); }

    public Worker(int[] array, int startIndex, int endIndex) {
        this.array = array; this.startIndex = startIndex; this.endIndex = endIndex;
    }

    @Override
    public void run() {
        for (int i = startIndex; i < endIndex; i++)
            sum.addAndGet(array[i]);
    }
}
```

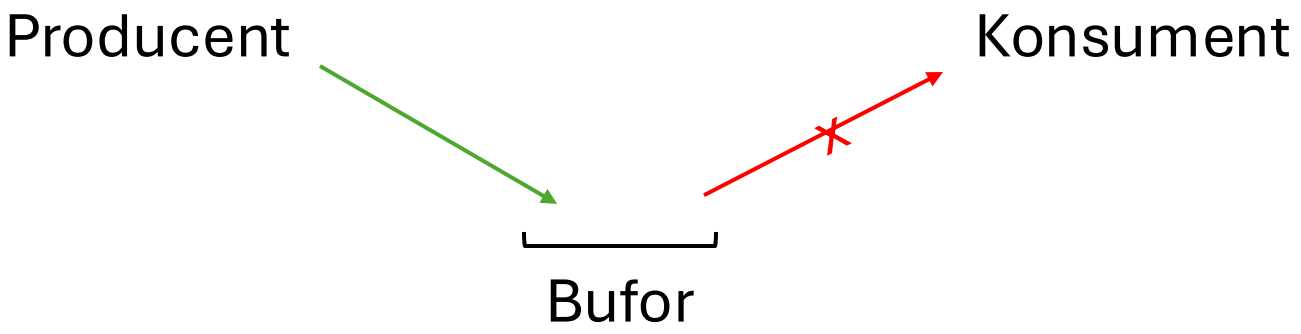
```
100000
```

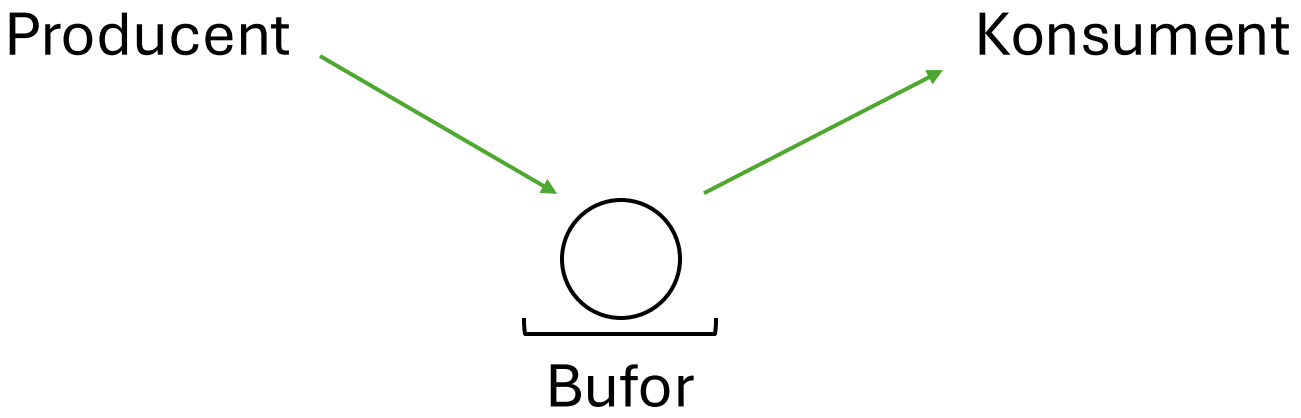
Sumy cząstkowe

```
class Worker implements Runnable {
    static private AtomicInteger sum = new AtomicInteger(0);
    private int partSum = 0;

    /* ... */

    @Override
    public void run() {
        for (int i = startIndex; i < endIndex; i++)
            partSum += array[i];
        sum.addAndGet(partSum);
    }
}
```

```
100000
```

Problem producenta i konsumenta





Producent i konsument

```
class Producer extends Thread {
   private Buffer buffer;

   public Producer(Buffer buffer) {
      this.buffer = buffer;
   }

   @Override
   public void run() {
      try {
         int i = 0;
         while (true) {
            buffer.produce(i++);
            Thread.sleep(1000);
         }
      } catch (InterruptedException e) {
         e.printStackTrace();
      }
   }
}
```

```
class Consumer extends Thread {
   private Buffer buffer;

   public Consumer(Buffer buffer) {
      this.buffer = buffer;
   }

   @Override
   public void run() {
      try {
         while (true) {
            buffer.consume();
            Thread.sleep(1500);          }
      } catch (InterruptedException e) {
         e.printStackTrace();
      }
   }
}
```

Współdzielony bufor

```
class Buffer {
    private LinkedList<Integer> queue = new LinkedList<>();
    private int capacity;

    public Buffer(int capacity) { this.capacity = capacity; }

    public synchronized void produce(int item) throws InterruptedException {
        while (queue.size() == capacity) wait();
        queue.add(item);
        System.out.println("Produced: " + item);
        notify();
    }

    public synchronized int consume() throws InterruptedException {
        while (queue.isEmpty()) wait();
        int consumedItem = queue.remove();
        System.out.println("Consumed: " + consumedItem);
        notify();
        return consumedItem;
    }
}
```

Jednoelementowy bufor

```
public static void main(String[] args) {
    Buffer buffer = new Buffer(1);
    Producer producer = new Producer(buffer);
    Consumer consumer = new Consumer(buffer);

    producer.start();
    consumer.start();
}
```

```
Produced: 0
Consumed: 0
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Produced: 6
Consumed: 6
Produced: 7
Consumed: 7
```

Wieloelementowy bufor

```
public static void main(String[] args) {
    Buffer buffer = new Buffer(5);
    Producer producer = new Producer(buffer);
    Consumer consumer = new Consumer(buffer);

    producer.start();
    consumer.start();
}
```

```
Produced: 0
Consumed: 0
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 3
Produced: 5
Consumed: 4
Produced: 6
Produced: 7
Consumed: 5
Produced: 8
Consumed: 6
```

Pule wątków

Runnable jako interfejs funkcyjny

```
int[] ints = new int[100000];
Arrays.fill(ints, 1);

Thread thread = new Thread(() -> {
    int sum = 0;
    for (int num : ints)
        sum += num;
    System.out.println(sum);
});
thread.start();
```

```
100000
```

Executor

```
int[] ints = new int[100000];
Arrays.fill(ints, 1);

Executor executor = Executors.newSingleThreadExecutor();
executor.execute(() -> {
    int sum = 0;
    for (int num : ints)
        sum += num;

    System.out.println(sum);
});
```

```
100000
```

ExecutorService

```
int[] ints = new int[100000];
Arrays.fill(ints, 1);

ExecutorService executor = Executors.newSingleThreadExecutor();
executor.submit(() -> {
    int sum = 0;
    for (int num : ints)
        sum += num;

    System.out.println(sum);
});

executor.shutdown();
```

```
100000
```

Szablon Future

```
int[] ints = new int[100000]; Arrays.fill(ints, 1);

ExecutorService executor = Executors.newSingleThreadExecutor();
Future<Integer> future = executor.submit(() -> {
    int sum = 0;
    for (int num : ints)
        sum += num;
    return sum;
});

try {
    int sum = future.get();
    System.out.println(sum);
} catch (Exception e) {
    e.printStackTrace();
}

executor.shutdown();
```

```
100000
```

Oczekiwanie na wynik

```
int[] ints = new int[100000]; Arrays.fill(ints, 1);

ExecutorService executor = Executors.newSingleThreadExecutor();
Future<Integer> future = executor.submit(() -> {
    int sum = 0;
    for (int num : ints)
        sum += num;
    return sum;
});

while(!future.isDone())
    System.out.print(".");

try { int sum = future.get(); System.out.println(sum);
} catch (Exception e) { e.printStackTrace(); }

executor.shutdown();
executor.awaitTermination(5, TimeUnit.SECONDS);
```

```
.............................100000
```

Podział zadania na wątki

```
int[] ints = new int[100000];
Arrays.fill(ints, 1);

int numThreads = Runtime.getRuntime().availableProcessors();
ExecutorService executor =
Executors.newFixedThreadPool(numThreads);
int chunkSize = ints.length / numThreads;

Future<Integer>[] futures = new Future[numThreads];

int sum = 0;
for (Future<Integer> future : futures) {
    try {
        sum += future.get();
    } catch (InterruptedException | ExecutionException e) {
        e.printStackTrace();
    }
}

System.out.println(sum);
```

```
for (int i = 0; i < numThreads; i++) {
      final int startIndex = i * chunkSize;
      final int endIndex =
          (i == numThreads - 1) ?
          ints.length : (i + 1) * chunkSize;
      futures[i] = executor.submit(() -> {
          int sum = 0;
          for (int j = startIndex; j < endIndex;
                                              j++)
              sum += ints[j];

          return sum;
      });
  }
```

```
100000
```